

DATA423-26S1 Assignment 2

20% of your final grade

Summary

This assignment has two parts, coding a shiny app and producing a report.

Recommendation: It is highly recommended to install the latest versions of the R packages you use.

Disclaimer: The data used in this assignment is not genuine; its has been artificially constructed to have interesting characteristics and challenges embedded in it.

Part 1 - Coding

Create a Shiny app using **RStudio**. Load the supplied comma separated variables (csv) file and use Shiny to:

1. Summarise and visualise the data and perform Exploratory Data Analysis. Make good use of controls. This is a minor part of the assignment so borrow from assignment 1 if you can.
2. From this evidence and the supplied background (see below) develop a strategy to deal with missing values. The strategy can change for different variables.
3. From this evidence and the supplied background develop a strategy to deal with outliers. The strategy can change for different variables.
4. Develop a pre-processing strategy for things like centring and scaling.
5. Implement these strategies using a “recipes” based data processing pipeline.
6. Develop a tuned **glmnet** model and visualise its test performance. Document the model’s optimal hyper-parameters. Note: **DO NOT** explore other methods for this assignment - just **glmnet**.
7. Identify any residual outliers. Think about how to show the train and test residuals.

The submission should be a set of files: **ui.R**, **server.R** and **global.R** that we should be able to run and grade (without needing to make any changes). Submit these files as a compressed **ZIP** file.

Part 2 - Report

Write a report on your modelling. Include appropriate images from your shiny app.

1. Discuss the data and any curious features that you noticed. Record the issues you would have followed up with a domain expert, were one available.
2. Document and justify your various strategies using words (rather than code).
3. Research the **glmnet** method and briefly explain this method in your report.
4. Document your **glmnet** model’s theoretical performance on unseen data.

Your report should have at least the following sections:

- Data Description
- Outlier Strategy
- Missingness Strategy
- Pre-Processing
- GLMNET Explanation
- Modelling
- Residual Analysis

Hint: All your figures and tables must be numbered and have a detailed description. Each figure or table must be referenced in your report. If not, why is it there?

Hint: Avoid using bullet points and numbered lists. Write using sentences and paragraphs.

Submit your report as a PDF, this must be submitted separately from the ZIP file.

The Background

Covid-19 data, all measurements are as at 2019. The supplied CSV contains the following variables:

CODE	Anonymised state or country
GOVERN_TYPE	Type of government: "STABLE DEM", "UNSTABLE DEM", "DICTATORSHIP", "OTHER"
POPULATION	Total population
AGE25_PROPTN	The proportion of the population that is at or below 25
AGE_MEDIAN	The median age of the population
AGE50_PROPTN	The proportion of the population that is at or above 50
POP_DENSITY	The population density
GDP	The Gross National Product
INFANT_MORT	The infant mortality rate
DOCS	The number of doctors per 10,000
VAX_RATE	The mean vaccination rate for Covid-19
HEALTHCARE_BASIS	Type of healthcare system "INSURANCE", "PRIVATE", "FREE"
HEALTHCARE_COST	Healthcare costs per person where applicable
DEATH_RATE	The projected death rate (across ten years)
OBS_TYPE	The allocation to test or train

The outcome variable is the **DEATH_RATE**.

The Details

Steps

1. Create a shell of a Shiny app. Plan whether you want a side-panel/main layout or a fluid-page layout or something more ambitious. Design how the user would progressively get more information by interacting with the page. Try to avoid long pages - instead use a tabset to control your navigation through the charts. You can continue developing your app from assignment 1 if you feel this is a good starting point. Add **your name** to the title part of the **UI** so it is clear to see whose app is running.

2. Identify all possible missing value placeholders eg ' ', 'na', 'N/A', -1, -99 etc.

3. Place the CSV file in the same location as the **ui.R**, **server.R** files. Load the CSV file using something like:

```
dat <- read.csv('data.csv', header = TRUE, na.strings = c('NA', 'N/A'), stringsAsFactors = TRUE)
```

4. Additionally replace numeric missing values with NA using something like:

```
dat[dat == -1] <- NA
```

5. Identify any categorical missing values that are “Not Applicable” and create new levels for these values using something like:

```
# convert away from factor
dat$cat <- as.character(dat$cat)
data$cat[is.na(data$cat)] <- "none"
# convert back to factor
dat$cat <- as.factor(dat$cat)
```

6. Identify any numeric missing values that are “Not Applicable” and create new levels for these values using something like:

```
# create a shadow variable
dat$num_shadow <- as.numeric(is.na(data$num))
# Assign missing to zero
dat$num[is.na(dat$num)] <- 0
```

7. Provide a set of EDA visualisations of the data set. Make a note of any curious things and add these to your report. Use lots of controls to vary the behaviour and scope of the visualisations. Remember to align the visualisation style to the variable type.
8. Identify any excessively missing variables for some threshold. Remove these.
9. Identify any excessively missing observations for some threshold. Remove these.
10. Determine if the missingness has pattern. Try something like:

```
library(rpart)
library(rpart.plot)
dat$MISSINGNESS <- apply(X = is.na(data), MARGIN = 1, FUN = sum)
tree <- train(MISSINGNESS ~ . - CODE, -OBS_TYPE,
              data = covid,
              method = "rpart",
              na.action = na.rpart)
rpart.plot(tree$finalModel,
            main = "TUNED: Predicting the number of missing variables in an observation",
            roundint = TRUE,
            clip.facs = TRUE)
```

11. Create a test - train split. Something like:

```
train ← dat[dat$OBS_TYPE == "Train",]
test ← dat[dat$OBS_TYPE == "Test"]
```

12. Develop a recipe-based processing pipeline. Something like:

```
# id is not a predictor
# obs_type is not a predictor
rec ← recipes::recipe(Target ~ ., data = dat)%>%
  update_role("ID", new_role = "id")%>%
  update_role("OBS_TYPE", new_role = "split")%>%
  step_knnimpute(all_predictors(), neighbors = 5)%>%
  step_center(all_numeric_predictors())%>%
  step_scale(all_numeric_predictors())%>%
  step_dummy(all_nominal_predictors())
```

13. Feed your recipe into a training operation that will optimise the hyperparameters of **glmnet** by resampling the train data. Do some research on the **caret** package. Choose a suitable metric to use in evaluating the model. Something like:

```
library(caret)
library(glmnet)
model ← caret::train(rec, data = train, method = "glmnet")
```

14. Generate the predictions for the test cases. Generate an appropriate visualisation for these predictions.
15. Display the test-RMSE statistic.
16. Generate a residual box-plot for the test data, the train data and both the test & train data and label the outliers based upon a slider for the IQR-multiplier.

Considerations

Here are some things to consider:

Interactive

The app should allow you to choose your strategy. By trying different strategies it should be possible to quantify whether they improve the model or not. It is not sufficient to just hard-code your optimal decisions.

How?

How can you make an app that allows your variable-missingness-threshold (for example) to affect your choice of whether to centre and scale? The key to this is to use **reactive expressions**.

Reactive expressions

The code below is a reactive expression that puts the data through a variable-missingness cleaning process:

```
getClean ← reactive({  
  #this is another reactive expression  
  d ← getData()  
  #process columns  
  vRatio ← apply(d, 2, pMiss)  
  d ← d[, vRatio < input$VarThresh]  
  #process rows  
  oRatio ← apply(d, 1, pMiss)  
  d ← d[oRatio < input$ObsThresh,]  
  d  
})
```

Whenever the function **getClean()** is called in Shiny code, the latest values of the **VarThresh** and **ObsThresh** sliders are used to generate the data.

Strategies

It may be necessary to have a cascade of reactive expressions that perform, in an appropriate sequence, the various strategies needed to optimally clean the data for a **glmnet** model.

Marking

This assignment is worth 20% of your final grade.

We mark the assignment out of 100 with 50% of the mark for the Shiny app and 50% for the report.

If you use an LLM or equivalent to aid your report writing and/or coding please acknowledge this in your report.

The Shiny App should run without errors. Test that it does.

Your code, and the use of your app, should allow exploration of different strategies.

The PDF document should show consistent and correct thinking about outliers, missing data, pre-processing steps and the assessment of a model.

The order of steps in the processing pipeline is important.

Good Luck!

The End